

基于 STM32F103ZE 的运行时变量监控与在线调参系统设计

《嵌入式系统与设计》课程设计报告

开发平台：正点原子战舰 STM32F103ZE

通信方式：CH340 USB 串口 / USART1

PC 端工具：XCOM V2.8 串口调试助手

最终固件：atk_f103_nolcd.uvprojx

摘要

针对嵌入式系统调试过程中运行时变量难以实时观察、参数修改需要反复编译下载的问题，本文设计并实现了一种基于 STM32F103ZE 的运行时变量监控与在线调参系统。系统以正点原子战舰开发板为硬件平台，利用 USART1 经 CH340 USB 串口与 PC 端 XCOM V2.8 串口调试助手通信，在 STM32 端维护可枚举、可读写的变量表，支持 PING、LIST、GET、SET 等 ASCII 文本命令。演示中通过 GET temp_x10 读取模拟温度、SET alarm_limit 280 在线修改报警阈值、GET alarm 读取报警状态、SET beep_en 0 在线关闭蜂鸣器，LED 与蜂鸣器产生板端报警反馈。系统模拟并复现了 STM32CubeMonitor 的核心思想：在目标程序持续运行过程中读取和修改关键变量，从而降低调试成本并提高参数整定效率。受 Keil 非商业版本代码大小限制，最终采用 no-LCD 瘦身版本进行稳定演示，LCD 接口已预留但未启用。

关键词：STM32F103ZE；USART；XCOM；运行时变量监控；在线调参；蜂鸣器报警

1 绪论

1.1 设计背景

课件资料中介绍的 STM32F103 属于基于 Cortex-M3 内核的 32 位微控制器，具有哈佛处理器架构、NVIC、SysTick、片上 Flash/SRAM、GPIO、USART、定时器等丰富外设[1-4]。在实际嵌入式开发中，许多控制参数和状态变量只能通过断点、串口 printf 打印或 JTAG/SWD 仿真器观察。如果每次调参都要修改代码、重新编译、下载和复位，调试效率将受到严重影响。

STMicroelectronics 推出的 STM32CubeMonitor 一类工具可以在目标运行时查看和修改变量[8]，但其通常依赖调试器（ST-LINK/J-LINK）和复杂的工程配置，对硬件环境和软件许可有一定要求。因此，本设计尝试以更低成本的方式复现其核心功能：仅使用普通 USART 串口和 PC 端串口助手，实现运行时变量的双向读写。

1.2 设计目标

- 实现 PC 端通过 CH340 USB 串口（USART1）向 STM32 发送文本命令，读取运行时变量。
- 实现 PC 端在线修改可写变量（如报警阈值、蜂鸣器使能），无需重新编译下载。
- 利用 LED 和蜂鸣器完成板端报警反馈，直观展示在线调参对系统行为的影响。
- 设计清晰的变量表和 ASCII 文本协议，使系统易于理解、测试和扩展到其他工程。
- 形成报告、固件代码、设计图、测试记录、截图与视频等完整课程设计提交材料。

2 需求分析与总体方案

2.1 功能需求

需求编号	功能	实现方式
F1	PC 与 STM32 通信	XCOM V2.8 + CH340 USB 串口 + USART1, 115200 8N1, ASCII 文本协议, 勾选"发送新行"
F2	连通性测试	发送 PING, 返回 OK PONG
F3	变量枚举	发送 LIST, 返回变量名、当前值、单位、读写属性和说明
F4	变量读取	发送 GET temp_x10, 返回模拟温度当前值
F5	在线调参	发送 SET alarm_limit 280, 在线修改报警阈值为 28.0°C
F6	报警状态读取	发送 GET alarm, 返回当前报警状态
F7	板端报警反馈	报警触发后 LED1 (PE5) 点亮, 蜂鸣器 (PB8) 鸣响
F8	蜂鸣器在线关闭	发送 SET beep_en 0, 关闭蜂鸣器而报警状态保持不变
F9	KEY 扩展	按键驱动已接入, KEY0/KEY1 可调节报警阈值, KEY2 可切换蜂鸣器使能, 作为本地调参扩展
F10	LCD	LCD 接口已预留, Board_LcdShowStatus 为空桩函数, 最终 no-LCD 版本未启用

2.2 总体结构

系统由 PC 端 XCOM V2.8 串口调试助手、CH340 USB 转串口链路、STM32F103ZE 主控及板载 LED/BEEP/KEY 等外设构成，如图 1 所示。

系统总体结构图

基于 STM32F103ZE 的运行时变量监控与在线调参系统（最终 no-LCD 演示版本）

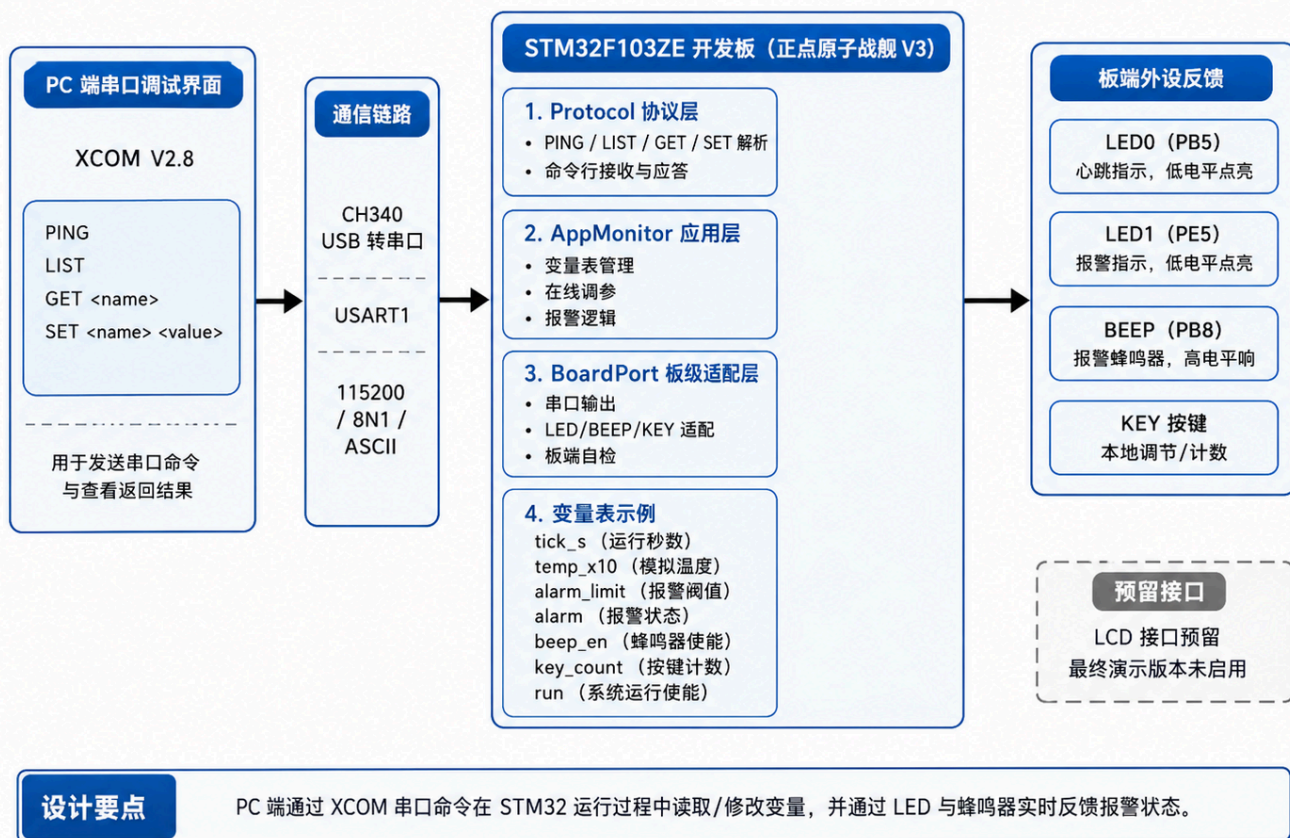


图 1 系统总体结构图

PC 端 XCOM V2.8 通过 CH340 USB 串口经 USART1（PA9 发送 / PA10 接收）与 STM32F103ZE 通信。STM32 端接收字符后, 经由协议层（monitor_protocol.c/h）完成命令解析, 再由应用层（app_monitor.c/h）访问变量表并执行报警逻辑, 最终通过板级适配层（board_port.c/h）控制 LED / BEEP / KEY 等外设。

3 硬件设计

硬件平台为正点原子战舰 STM32F103ZE 开发板。STM32F103ZE 采用 ARM Cortex-M3 内核, 课件《Lec6》指出 STM32F103 具有 512 KB Flash、64 KB SRAM、多组总线、GPIO、USART、定时器和丰富外设; 战舰板还集成了 CH340 USB 转串口、按键、蜂鸣器、LED 和调试下载接口, 适合作为本设计的实验平台, 相关硬件连接可参考正点原子开发指南和开发板原理图[6-7]。

本项目的硬件连接关系如图 2 所示。PC 端通过两条链路与开发板连接: 一条是 CH340 USB 转串口链路, 用于 XCOM 与 STM32 之间的运行时变量监控和在线调参; 另一条是 ST-LINK/V2 经 JTAG/SWD 连接开发板, 用于程序下载和必要的调试。最终演示中的命令交互均通过 CH340 / USART1 完成, ST-LINK 只承担下载和调试作用。

硬件连接与接口原理图

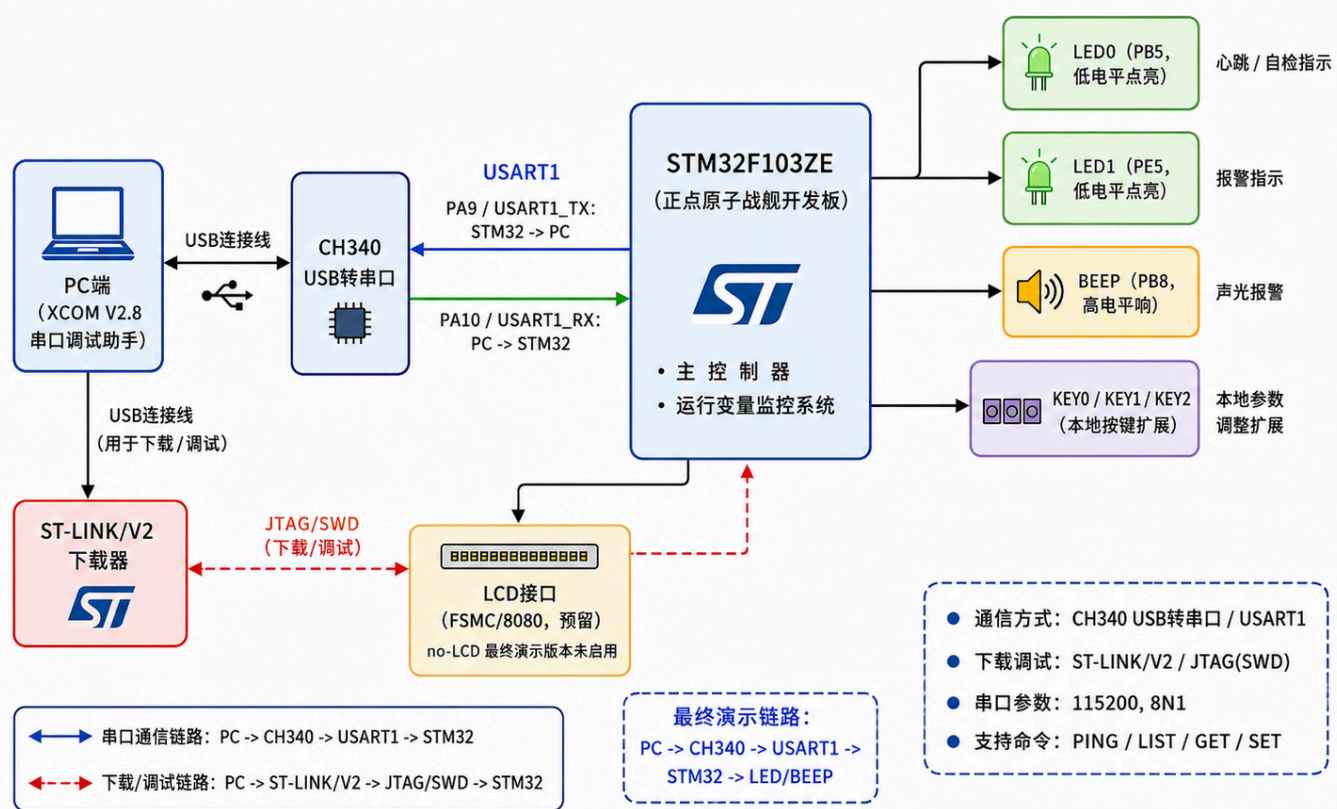


图 2 硬件连接与接口原理图

外设	引脚/接口	本设计用途
USART1_TX	PA9	STM32 向 PC 返回命令响应
USART1_RX	PA10	PC 向 STM32 发送命令
CH340	USB 转串口	PC 与开发板之间的串口桥接
LED0	PB5	低电平点亮，系统自检时使用，运行时作为心跳指示
LED1	PE5	低电平点亮，报警状态指示
BEEP	PB8	高电平鸣响，报警蜂鸣器
KEY	以 key.h 定义为准 (KEY0 / KEY1 / KEY2)	驱动已接入，可作为本地调参扩展，最终视频未重点演示按键调参
LCD	FSMC/8080 并口 (预留)	接口已预留，Board_LcdShowStatus 为空桩函数，最终 no-LCD 版本未启用

由于不同批次开发板或例程可能存在宏定义差异，代码采用 BoardPort 适配层封装底层驱动。实际移植时只需要把 BoardPort 中的 LED、KEY、BEEP、USART 函数映射到正点原子例程中的函数即可，不影响上层协议和应用逻辑。

4 软件设计

4.1 软件分层

层次	文件/工具	职责
PC 端串口调试界面	XCOM V2.8	发送命令、显示返回结果
协议层	monitor_protocol.c/h	串口行缓冲、PING/LIST/GET/SET 命令解析、变量查找与读写响应
应用层	app_monitor.c/h	变量表定义、模拟温度生成、报警判断、周期任务调度
板级适配层	board_port.c/h	USART 输出、LED/BEEP/KEY 底层适配，屏蔽具体板卡驱动差异
底层驱动	HAL、USART、GPIO、BSP	芯片外设初始化与控制[1-2,6]

各层之间通过明确的函数接口通信：协议层只通过 `mon_puts_fn` 输出字符串，不直接操作 USART 寄存器；应用层通过 `Board_LedSet`、`Board_BeepSet`、`Board_KeyScan` 等接口访问板载外设。这种分层使得协议逻辑与具体硬件解耦，便于移植到其他 STM32 平台。

4.2 变量表机制

变量表是本设计的核心。每个变量条目包含变量名、类型、内存地址、读写权限、单位和说明。协议层并不关心变量来自哪个模块，只通过地址读写对应内存。这种设计与调试器观察变量的思想相似，但实现方式更轻量：用户只需要把希望监控的变量加入数组，即可被 PC 端动态读取或修改。

变量名	含义	权限	单位/类型
<code>tick_s</code>	系统运行秒数	只读	s
<code>temp_x10</code>	模拟温度，放大 10 倍	可写	0.1°C
<code>alarm_limit</code>	报警阈值	可写	0.1°C
<code>adc_mv</code>	模拟 ADC 电压	可写	mV
<code>led_period</code>	LED0 心跳周期	可写	100ms
<code>beep_en</code>	蜂鸣器使能	可写	bool
<code>alarm</code>	报警状态	只读	bool
<code>key_count</code>	按键计数	只读	cnt
<code>run</code>	系统运行使能	可写	bool

其中 `temp_x10`、`alarm_limit`、`beep_en`、`alarm` 为最终演示中实际使用的核心变量，`adc_mv`、`led_period`、`key_count`、`run` 为扩展变量，可按需通过 `GET/SET` 访问。最终演示视频中，`temp_x10` 仅作为模拟温度读取变量进行展示，在线修改重点为 `alarm_limit` 和 `beep_en`。

4.3 串口协议

系统采用 ASCII 文本协议，串口参数为 115200 波特率、8 数据位、无校验、1 停止位，XCOM 勾选"发送新行"以自动追加回车换行。相比二进制协议，文本协议效率略低，但调试友好，串口助手可直接输入命令，适合课程设计演示和问题定位。命令以 CR/LF 结束（即 \r 或 \n），协议层在收到完整行后执行解析和响应。

命令	功能	示例	示例响应
PING	连通性测试	PING	OK PONG
LIST	列出变量表	LIST	VAR temp_x10 250 0.1℃ W virtual_temperature 等，最后返回 OK LIST_END
GET <name>	读取变量	GET temp_x10	DATA temp_x10 286 0.1℃
SET <name> <value>	修改变量	SET alarm_limit 280	OK SET alarm_limit 280
HELP	显示帮助	HELP	OK LIST GET <name> SET <name> <value> PING

最终演示命令顺序与视频完全一致：

```
PING
LIST
GET temp_x10
SET alarm_limit 280
GET alarm
SET beep_en 0
```

4.4 软件流程

软件流程与协议状态机

基于 STM32F103ZE 的运行时变量监控与在线调参系统（最终 no-LCD 演示版本）

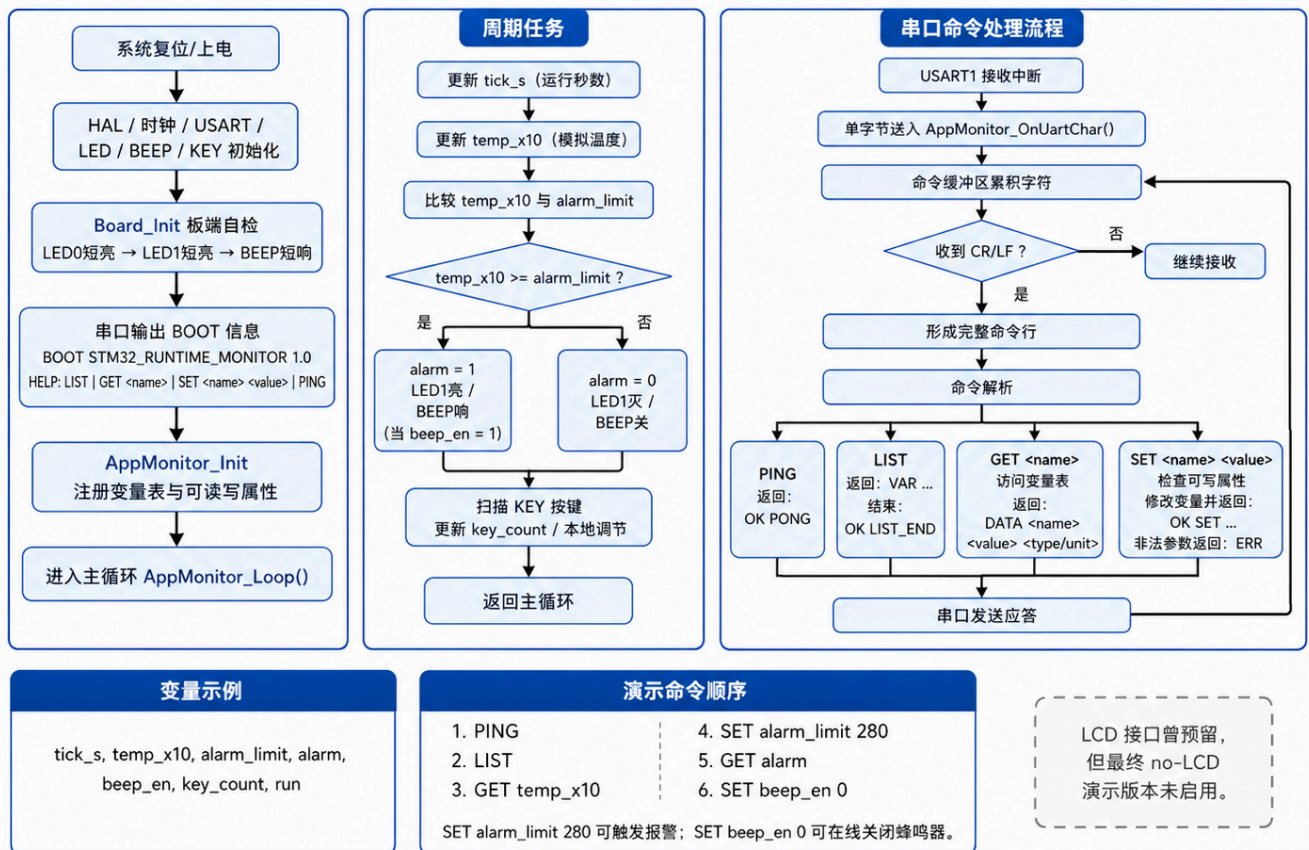


图 3 软件流程与协议状态机

复位后, 系统执行 `Board_Init()` 完成 GPIO 自检, 依次点亮 LED0、LED1、BEEP 以验证外设正常, 并通过串口输出 `BOARD_SELFTEST_DONE`。随后 `AppMonitor_Init()` 调用 `Monitor_Init()` 注册变量表和输出函数, 串口输出 `BOOT_STM32_RUNTIME_MONITOR_1.0` 和帮助信息。

主循环调用 `AppMonitor_Loop()`, 其中完成协议处理、周期任务和必要延时。周期任务按约 100ms 节拍更新系统运行秒数、生成模拟温度、扫描按键、更新报警状态, 并通过 `BoardPort` 控制 LED/BEEP 输出。

USART1 接收中断将收到的字符交给 `AppMonitor_OnUartChar()`, 进而调用 `Monitor_OnChar()`。协议层维护命令行缓冲区, 遇到 CR/LF 结束符后调用 `Monitor_ProcessLine()` 进行解析: 匹配 `PING/LIST/GET/SET/HELP`, 访问变量表完成读取或写入, 最后通过 USART1 返回 `OK/DATA/ERR` 响应。

5 关键程序说明

下面给出几个核心设计片段, 完整代码见 `02_固件代码/RuntimeMonitor/`。

5.1 变量表定义

变量表将 C 变量和协议可见名称绑定起来, 写权限由 `writable` 字段控制。PC 端只能修改标记为可写的变量, `tick_s`、`alarm`、`key_count` 等只读变量被协议层拒绝写入, 避免误操作破坏系统状态。

```
static mon_var_t g_vars[] = {
    {"tick_s",      MON_VAR_U32,  &g_tick_s,      0, "s",
```

```

"system_seconds"},
    {"temp_x10",    MON_VAR_I32,  &g_temp_x10,          1, "0.1C",
"virtual_temperature"},
    {"alarm_limit", MON_VAR_I32,  &g_alarm_limit_x10,    1, "0.1C",
"alarm_threshold"},
    {"adc_mv",      MON_VAR_U16,  &g_virtual_adc_mv,     1, "mV",
"virtual_adc_value"},
    {"led_period",  MON_VAR_U16,  &g_led_period_100ms,   1, "100ms",
"led_toggle_period"},
    {"beep_en",     MON_VAR_U8,   &g_beep_enable,       1, "bool",
"beeper_enable"},
    {"alarm",       MON_VAR_U8,   &g_alarm,             0, "bool",
"alarm_state"},
    {"key_count",   MON_VAR_U32,  &g_key_count,         0, "cnt",
"key_press_count"},
    {"run",         MON_VAR_U8,   &g_run,               1, "bool",
"system_run_enable"},
};

```

5.2 main.c 中的初始化与循环接入

在 `main()` 函数中，外设初始化完成后调用 `AppMonitor_Init()` 注册变量表并输出启动信息，主循环通过 `AppMonitor_Loop()` 驱动周期任务和协议处理。

```

// 简化片段，完整代码见 02_固件代码/RuntimeMonitor/
int main(void)
{
    HAL_Init();

    /* 时钟、延时、USART1、LED/BEEP/KEY 等初始化保持正点原子原工程流程 */
    /* 例如系统时钟初始化、delay 初始化、USART1 初始化和板级外设初始化 */

    AppMonitor_Init();

    while (1)
    {
        AppMonitor_Loop();
    }
}

```

5.3 usart.c 中的串口接收接入

在 USART1 单字节接收完成回调中，将 `g_rx_buffer[0]` 交给 `AppMonitor_OnUartChar()`，随后重新启动下一次中断接收。这样 PC 端发送的每个字符都能进入协议层缓冲区，直到检测到 CR/LF 后解析完整命令。

```

// 简化片段，完整代码见 02_固件代码/RuntimeMonitor/
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart == &g_uart1_handle)

```

```
{
    AppMonitor_OnUartChar((char)g_rx_buffer[0]);
    HAL_UART_Receive_IT(&g_uart1_handle,
                        (uint8_t *)g_rx_buffer,
                        RXBUFFERSIZE);
}
}
```

5.4 BoardPort 适配层

BoardPort 将底板驱动封装成统一接口。`Board_BeepSet(1)` 驱动 PB8 输出高电平使蜂鸣器鸣响，`Board_LedSet(1, 1)` 驱动 PE5 输出低电平点亮 LED1。`Board_LcdShowStatus()` 在 no-LCD 版本中为空桩函数，参数均被 `(void)` 忽略，保留接口以便日后启用 LCD 时无需修改应用层代码。

6 调试过程与结果分析

6.1 调试过程

系统调试经历了以下关键步骤：

1. **Keil 环境搭建**：安装 MDK-ARM 及 STM32F1 Device Family Pack，导入 HAL 库和 CMSIS。Keil MDK-ARM 是本项目使用的主要编译和下载环境，工程设置与编译器选项可参考 MDK-ARM 文档[5]。
2. **编译器与 LTO 问题处理**：ARM Compiler 6 环境下曾出现 LTO 相关链接错误，当前 Keil 版本不支持该优化选项。关闭 Link Time Optimization 后，工程能够正常编译。
3. **正点原子串口实验跑通**：先验证原始串口收发例程能在战舰板上正常工作，确认 CH340 驱动和硬件连接无误。
4. **中文路径问题**：初期工程位于中文路径下导致编译错误，将整个工程迁移至英文路径后解决。
5. **加入协议层与应用层**：将 `monitor_protocol.c/h` 和 `app_monitor.c/h` 加入工程，构建变量表和命令解析逻辑。
6. **修改 main.c 和 usart.c**：调用 `AppMonitor_Init()` 和 `AppMonitor_OnUartChar()` 完成功能接入。
7. **XCOM 测试**：使用 XCOM V2.8 依次发送 BOOT 检查、PING、LIST、GET temp_x10、SET alarm_limit、GET alarm、SET beep_en 等命令，逐项验证协议响应正确性。
8. **加入 LED / BEEP / KEY**：通过 BoardPort 适配正点原子 BSP 的 GPIO 和按键驱动，完成板端报警反馈和按键扩展。
9. **LCD 版本代码大小超限**：初期曾尝试加入 LCD / FSMC / SRAM 等相关驱动，工程体积超过 Keil 非商业版本代码大小限制，出现代码大小超限链接错误，无法生成最终可烧录镜像。
10. **切换到 no-LCD 版本**：创建 `atk_f103_nolcd.uvprojx` 工程，移除 LCD 相关驱动和 FSMC 初始化，将 `Board_LcdShowStatus()` 改为空桩函数，编译通过并成功下载。
11. **直接 GPIO 自检确认**：`Board_Init()` 中依次驱动 LED0、LED1、BEEP 并串口输出自检信息，确保外设物理连接正常。
12. **最终视频演示**：按最终命令序列完整演示，所有功能通过。

6.2 最终测试表

序号	测试命令 / 动作	期望现象	结果
1	复位上电	XCOM 输出 <code>BOOT STM32_RUNTIME_MONITOR 1.0</code> 、HELP 信息和自检提示	通过
2	<code>PING</code>	返回 <code>OK PONG</code>	通过
3	<code>LIST</code>	返回所有变量条目及 <code>OK LIST_END</code>	通过
4	<code>GET temp_x10</code>	返回 <code>DATA temp_x10 <value> 0.1℃</code> ，值在 240~420 之间波动	通过
5	<code>SET alarm_limit 280</code>	返回 <code>OK SET alarm_limit 280</code> ，报警阈值修改为 28.0℃	通过
6	<code>GET alarm</code>	模拟温度高于阈值时返回 <code>DATA alarm 1 bool</code>	通过
7	板端观察	LED1 (PE5) 点亮，蜂鸣器 (PB8) 鸣响，产生报警反馈	通过
8	<code>SET beep_en 0</code>	蜂鸣器关闭， <code>GET alarm</code> 仍返回 1，LED1 仍点亮	通过

7 创新性与实用性

本设计的得意之处在于：将嵌入式程序内部变量抽象为可枚举、可读写的变量表，使 PC 端能够在程序持续运行时通过串口完成观察与调参，而不依赖 JTAG 调试器。系统没有把开发板简单做成 LED 流水灯或固定功能演示，而是把“调试过程本身”设计成一个嵌入式应用。通过变量表把目标程序内部状态开放给 PC 端，使开发者可以在运行中观察、修改参数并即时看到板端反馈。

与传统串口 `printf` 调试相比，本系统不是单向输出日志，而是具备双向读写能力——既能 `GET` 观察变量，也能 `SET` 在线修改；与完整调试器方案相比，本系统硬件成本低，只需要普通串口和串口助手即可工作；与依赖调试接口的运行时监控方案相比，本系统的变量查看和在线调参过程不依赖 ST-LINK 调试通道，而是通过 CH340 / USART1 串口完成；ST-LINK/V2 仅用于程序下载和必要调试。该方案可迁移到温控、电机 PID、传感器标定等需要快速参数整定的真实工程中。

8 心得体会

通过本次课程设计，我对嵌入式系统的理解从“会点亮外设”进一步转向“如何组织一个可维护、可调试、可扩展的系统”。Cortex-M3 / STM32 的外设很多，如果直接在 `main` 函数里堆代码，短期能跑，后期很难扩展。本设计采用协议层、应用层、板级适配层分离的方法，使串口协议和具体硬件驱动解耦。这让我认识到，嵌入式软件同样需要清晰的接口和模块边界。

在线调参功能让我体会到调试工具的重要性。很多控制系统不是代码写完就结束，而是需要反复观察变量、调整阈值和验证边界情况。如果能在运行中直接修改变量，就能显著减少编译下载次数，提高实验效率。

调试过程中遇到了一些实际问题：Keil 非商业版本存在代码大小限制，LCD 驱动代码较大导致编译失败，最终通过瘦身方案解决；中文路径曾导致编译报错，工程需置于全英文路径；ARM Compiler 6 环境下曾出现

LTO 相关链接错误，当前 Keil 版本不支持该优化选项，最终通过关闭 Link Time Optimization 解决。这些问题让我认识到，实际嵌入式开发中工具链配置和环境管理同样重要。通过逐步排查、分模块验证，最终完成了稳定可演示的系统。

9 成员贡献说明

成员	主要贡献	比例
本人	选题分析、系统方案设计、协议设计、固件移植、BoardPort 适配、XCOM 串口调试、LED/BEEP 报警调试、演示视频录制、报告与提交材料整理。	100%

10 结论

本文完成了基于 STM32F103ZE 的运行时变量监控与在线调参系统设计。系统以 USART1 + CH340 为通信链路，以 XCOM V2.8 为 PC 端调试界面，实现了 PING 连通性测试、LIST 变量枚举、GET 变量读取、SET 在线修改等核心功能。最终演示中通过 `GET temp_x10` 读取模拟温度、`SET alarm_limit 280` 在线修改报警阈值、`GET alarm` 读取报警状态、`SET beep_en 0` 在线关闭蜂鸣器，LED1 和蜂鸣器产生明确的板端报警反馈。

受 Keil 非商业版本代码大小限制，最终采用 atk_f103_nolcd 瘦身版本进行稳定演示，LCD 接口已预留但未启用；KEY 驱动已接入，可作为本地调参扩展。项目实现了题目"主机接口软件"所要求的低成本运行时变量监控与在线调参目标，满足课程设计对外设综合应用、代码设计、调试过程和结果展示的要求。

后续可进一步扩展为图形化上位机、二进制高效协议、CRC 校验、变量分组、波形绘制，以及对真实传感器或 PID 控制系统的在线调参。

参考文献

[1] STMicroelectronics. STM32F103xC/D/E Datasheet (DS5792) [Z].

[2] STMicroelectronics. RM0008 Reference Manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs [Z].

[3] ARM Limited. ARM Cortex-M3 Processor Technical Reference Manual [Z].

[4] Joseph Yiu. The Definitive Guide to the ARM Cortex-M3 [M]. Elsevier.

[5] Keil. MDK-ARM Getting Started Guide [Z].

[6] 正点原子. STM32F1 开发指南 HAL 库版本 [Z].

[7] 正点原子. WarShip STM32F1 V3 原理图 [Z].

[8] STMicroelectronics. STM32CubeMonitor User Manual (UM2402) [Z].